	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch:
	Name: Shivam Kumar Gautam Reg. No: 2503A52457
	Department: Computer Science and AI Semester: I
	AI Assisted Coding-Assignment Date: 10/07/2026

TASK 1 – Improve Readability

Original Code:

```
a=80
b=70
c=90
x=(a+b+c)/3
if x>=50:
  print("pass")
else:
  print("fail")
```

Prompt:

Use meaningful variable names and create a function such as calculate_average() then improve formatting and readability after this run both versions and verify the same result and explain the improvements.

Code:

```
def calculate_average(scores):
  """Return the arithmetic mean of a non-empty list of scores."""
  if not scores:
    raise ValueError("scores cannot be empty")
  return sum(scores) / len(scores)
```

```
scores = [80, 70, 90]
function_average = calculate_average(scores)

# This is the second version, calculated directly instead of through the function.
a = 80
b = 70
c = 90
direct_average = (a + b + c) / 3

# Verify that both versions produce the same result before printing it.
assert function_average == direct_average
print(f"Function average: {function_average:.2f}")
print(f"Direct average: {direct_average:.2f}")
print("Both versions produce the same result.")

if function_average >= 50:
   print("pass")
else:
   print("fail")
```

output:

```
Function average: 80.00
Direct average: 80.00
Both versions produce the same result.
Pass
```

TASK 2 – Remove Repeated Code

Original code:

```
name1 = "Arun"
marks1 = 85
if marks1 >= 50:
  print(name1, "Pass")
else:
  print(name1, "Fail")

name2 = "Priya"
marks2 = 72
if marks2 >= 50:
  print(name2, "Pass")
else:
  print(name2, "Fail")
```

prompt:

Identify duplicated logic and Refactor the repeated logic into a reusable function after this test the function with at least four students then explain why removing duplication improves maintainability and verify that output remains correct.

```
# The pass/fail conditional was duplicated for each student. Keeping it in one
# function makes updates easier and ensures results are calculated consistently.
def student_result(name, marks):
  status = "Pass" if marks >= 50 else "Fail"
  return f"{name} {status}"

students = [("Arun", 85), ("Priya", 72), ("Dev", 45), ("Maya", 50)]
expected_results = ["Arun Pass", "Priya Pass", "Dev Fail", "Maya Pass"]
results = [student_result(name, marks) for name, marks in students]
```

```
# Check that the refactored function preserves the expected output.

assert results == expected_results

for result in results:

   print(result)
```

output:

Arun Pass

Priya Pass

Dev Fail

Maya Pass

TASK 3 – Refactor a Long Conditional

Original Code:

```
age = 68
temperature = 39

if age >= 60 and temperature >= 38:
   print("High Priority")
elif age >= 60 and temperature < 38:
   print("Medium Priority")
elif age < 60 and temperature >= 38:
   print("Medium Priority")
else:
   print("Normal Priority")
```

prompt:

Simplify the conditional logic and create determine_priority() after this test at least four age/temperature combinations then Compare original and refactored logic and explain whether AI preserved the original behavior

Code :

```
def original_priority(age, temperature):
    """Apply the original, unsimplified priority rules."""
    if age >= 60 and temperature >= 38:
        return "High Priority"
    elif age >= 60 and temperature < 38:
        return "Medium Priority"
    elif age < 60 and temperature >= 38:
        return "Medium Priority"
    return "Normal Priority"

def determine_priority(age, temperature):
    """Apply the equivalent priority rules with simplified conditions."""
    if age >= 60 and temperature >= 38:
        return "High Priority"
    if age >= 60 or temperature >= 38:
        return "Medium Priority"
    return "Normal Priority"

test_cases = [(68, 39), (68, 37), (40, 39), (40, 37)]
all_match = True

for age, temperature in test_cases:
    original = original_priority(age, temperature)
    refactored = determine_priority(age, temperature)
    print(
```

```
        f"Age {age}, temperature {temperature}: "
        f"original={original}, refactored={refactored}"
    )
    all_match = all_match and original == refactored

print(
    "AI preserved the original behavior for all tested combinations."
    if all_match
    else "AI did not preserve the original behavior for all tested combinations."
)
```

Output:

Age 68, temperature 39: original=High Priority, refactored=High Priority

Age 68, temperature 37: original=Medium Priority, refactored=Medium Priority

Age 40, temperature 39: original=Medium Priority, refactored=Medium Priority

Age 40, temperature 37: original=Normal Priority, refactored=Normal Priority

AI preserved the original behavior for all tested combinations.

TASK 4 – Refactor Inefficient List Processing

Original code:

```
patient_ids = [101, 205, 309, 412, 518, 623]

target = 412
found = False

for patient_id in patient_ids:
    if patient_id == target:
        found = True
```

```
if found:
  print("Patient Found")
else:
  print("Patient Not Found")
```

prompt:

Improve readability and check whether a set or dictionary is better for frequent lookups and implement the justified suggestion then test existing and missing IDs and compare typical list and set lookup complexity.

Code:

```
patient_ids = [101, 205, 309, 412, 518, 623]

patient_id_set = set(patient_ids)


def find_patient(patient_id, patient_ids_to_check):
  """Return whether a patient ID exists in the supplied collection."""
  return patient_id in patient_ids_to_check


def test_patient_lookup(patient_id):
  """Test lookup behavior for an existing and a missing patient ID."""
  found_in_list = find_patient(patient_id, patient_ids)
  found_in_set = find_patient(patient_id, patient_id_set)


  if found_in_list and found_in_set:
    print(f"Patient {patient_id}: Found in list and set")
  elif not found_in_list and not found_in_set:
    print(f"Patient {patient_id}: Not found in list or set")
  else:
```

```
        print(f"Patient {patient_id}: Inconsistent lookup results")

# A list supports membership checks in O(n), while a set supports them in
# average O(1). A set is preferable when the same IDs are checked repeatedly;
# the list is more suitable when the collection is small or must preserve order.
for patient_id in (412, 999):
    test_patient_lookup(patient_id)

print("List lookup complexity: O(n)")
print("Set lookup complexity: average O(1)")
```

output:

```
Patient 412: Found in list and set
Patient 999: Not found in list or set
List lookup complexity: O(n)
Set lookup complexity: average O(1)
```

TASK 5 – Complete Legacy Code Review with AI

Original code:

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

total = 0
for s in students:
    total = total + s[1]

avg = total / len(students)

for s in students:
    if s[1] >= 50:
```

```
    print(s[0], "Pass")
  else:
    print(s[0], "Fail")

print("Average =", avg)
```

prompt:

Identify at least five improvements and Refactor using meaningful names and reusable functions then use suitable Python features when they improve clarity after this run both versions and verify equivalent results then record AI suggestions as accepted/rejected with reasons and identify the most useful refactoring change and justify it.

Code:

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

PASSING_MARKS = 50


def run_legacy_version(student_records):
  """Run the original loop-based logic and return results and average."""
  total_marks = 0
  student_results = []
  for student_name, marks in student_records:
    total_marks += marks
    if marks >= PASSING_MARKS:
      student_results.append((student_name, "Pass"))
    else:
      student_results.append((student_name, "Fail"))
  average_marks = total_marks / len(student_records)
  return student_results, average_marks
```

```
def calculate_average(student_records):
    """Return the average mark, rejecting an empty collection clearly."""
    if not student_records:
        raise ValueError("student_records cannot be empty")
    return sum(marks for _, marks in student_records) / len(student_records)


def get_student_result(student_name, marks):
    """Return one student's name and pass/fail status."""
    status = "Pass" if marks >= PASSING_MARKS else "Fail"
    return student_name, status


def process_students(student_records):
    """Return all student statuses and their average mark."""
    results = [
        get_student_result(student_name, marks)
        for student_name, marks in student_records
    ]
    return results, calculate_average(student_records)

# Run both implementations and check that the refactor preserves behavior.
legacy_results, legacy_average = run_legacy_version(students)
refactored_results, refactored_average = process_students(students)
assert refactored_results == legacy_results
assert refactored_average == legacy_average


for student_name, status in refactored_results:
```

```
    print(student_name, status)
print("Average =", refactored_average)
print("Legacy and refactored results are equivalent.")

# AI refactoring suggestions:
# Accepted — descriptive names replace s, total, and avg, making data roles clear.
# Accepted — separate reusable functions handle average, one result, and processing.
# Accepted — a named passing-mark constant avoids repeating the threshold literal.
# Accepted — sum() and a comprehension express aggregation and result creation clearly.
# Accepted — explicitly reject empty input with a useful ValueError message.
# Accepted — run both implementations and assert their outputs and averages match.
# Rejected — add a class/dataclass for each student; tuples already suit this small,
#          fixed input and a class would add structure without improving clarity.
# Most useful: extracting get_student_result centralizes pass/fail policy, so a
# future threshold or status-rule change has one implementation to update.
```

Output:

```
Arun Pass
Priya Fail
Rahul Passs
Meena Pass
Average = 65.75
Legacy and refactored results are equivalent.
```